

Reduced Precision Hardware Acceleration of Nested Multi-Level Monte Carlo using CPUs



Mathematical
Institute

MOHAMMAD SAIFULLAH

SUPERVISOR: PROF. MIKE GILES

Mathematical Institute, University of Oxford

September 2026



Oxford
Mathematics



The problem

- Pricing a financial option means estimating $\mathbb{E}[P]$ for a payoff P of an SDE that has no closed form solution. Plain Monte Carlo usually costs $O(\varepsilon^{-3})$ for RMS accuracy ε .
- MLMC [2, 3] writes it as a telescoping sum over a hierarchy of time grids, and costs $O(\varepsilon^{-2})$ when the level variance decays fast enough:

$$\mathbb{E}[P_L] = \sum_{\ell} \mathbb{E}[P_{\ell} - P_{\ell-1}], \quad \mathbb{V}[P_{\ell} - P_{\ell-1}] = O(2^{-\beta\ell}), \quad C_{\ell} = O(2^{\gamma\ell}).$$

The cost result needs $\beta > \gamma$. For Lipschitz payoffs, Milstein [11] gives $\beta \approx 2$ against Euler–Maruyama’s $\beta \approx 1$.

- Payoffs: a scalar Asian call, and European and Asian calls on a five-asset correlated basket.

The objective

- MLMC already reduces cost by putting most samples on coarse grids. The objective here is the further saving available from vectorisation on modern Intel CPUs, and in particular from FP16 against FP32. A 512-bit register holds 32 FP16 values against 16 FP32, so FP16 offers twice the data-level parallelism, but its unit roundoff is 2^{-11} against FP32's 2^{-24} , and β is sensitive to rounding.
- Prior work [7, 13, 14] treats Euler–Maruyama, so this thesis checks what reduced precision costs a nested Milstein MLMC estimator.
- Nested MLMC [5, 13] adds a second hierarchy, in precision. Each grid-level correction is split into a cheap FP16 term and a correction for what FP16 got wrong, giving two super-levels per grid level k :

$$\underbrace{Y_{2k} = \Delta \hat{P}_k^{\text{FP16}}}_{\text{cheap, many samples}} \quad \underbrace{Y_{2k+1} = \Delta P_k^{\text{FP32}} - \Delta \hat{P}_k^{\text{FP16}}}_{\text{correction, few samples}}$$

Result 1: the rounding-error model

Milstein step, bracketed by magnitude, with $S_n = O(1)$:

$$\Delta S_n = \underbrace{\sigma S_n \Delta W_n}_{O(\sqrt{h})} + \underbrace{\left(a S_n h + \frac{1}{2} \sigma^2 S_n (\Delta W_n^2 - h) \right)}_{O(h)}$$

Each FP16 step commits three rounding errors [4, 8]:

$$\hat{S}_{n+1} = \hat{S}_n + \Delta \hat{S}_n + \underbrace{\eta_n}_{O(\delta)} + \underbrace{\eta'_n}_{O(\delta\sqrt{h})} + \underbrace{\eta''_n}_{O(\delta h)}.$$

Squaring the three path errors over $N = 2^k$ steps, only η_n carries the level: it gives $N\delta^2 = 2^k\delta^2$, against δ^2 and $\delta^2 2^{-k}$, which are absorbed into the constants. Without compensation the level variance is then two competing terms,

$$\mathbb{V}[Y_{2k}] \approx \underbrace{C_1 h_k^2}_{\text{discretisation}} + \underbrace{C_2 2^k \delta^2}_{\text{rounding}}.$$

The first falls with k and the second grows, so the curve turns at the crossover where they balance,

$$k^* \approx \frac{2}{3} \log_2(\delta^{-1}), \quad \delta_{16} = 2^{-11} \Rightarrow k^* \approx 7.$$

A scaling estimate, not the integer cutoff: C_1, C_2 are payoff-dependent.

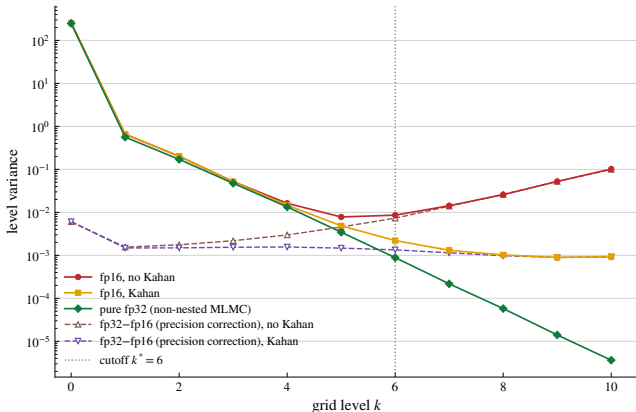
Result 1: compensated summation

Kahan compensated summation [10] keeps a running estimate c of what the previous add discarded, and folds it into the next increment:

$$\begin{aligned} y &\leftarrow \Delta S_n - c && \text{fold in what was lost} \\ t &\leftarrow \widehat{S}_n + y && \text{the rounded sum} \\ c &\leftarrow (t - \widehat{S}_n) - y && \text{recover this rounding} \\ \widehat{S}_{n+1} &\leftarrow t \end{aligned}$$

It reaches η_n , which is presented to the accumulator, but not η'_n or η''_n , which are committed inside the arithmetic forming ΔS_n . So it shrinks C_2 rather than removing it: the crossover moves to a larger k^* and the floor beyond it drops, but the decay is not restored indefinitely.

Basket European: AVX-512 fp16/fp32 precision comparison ($N = 20,000$)



Result 2

Level-0 basket European kernel, AVX-512 on a Xeon Gold 6538Y+, $N = 4.19 \times 10^6$, single thread.

	FP32	FP16 _K	Ratio
Time per path (ns)	25.8	12.5	2.07×
Cycles	1 446 M	701 M	2.06×
Clock frequency (GHz)	2.651	2.655	equal
FP-arith instructions	260.3 M	134.1 M	1.94×
L1 data-cache loads	658.0 M	126.4 M	5.20×
L1 data-cache stores	231.1 M	25.8 M	8.96×
L1 miss rate	0.05%	0.07%	
IPC (all)	1.52	1.28	
IPC (non-memory)	0.90	1.06	
Memory share of stream	40.5%	17.0%	

- Equal clock, so the speedup is entirely fewer cycles. The kernel issues half as many instructions, each running at the same speed.
- Memory falls far more than arithmetic does (5.20×
- FP32 retires more per cycle on the whole stream, but 40.5% of that stream is memory. Counting only non-memory instructions reverses it: FP32's higher IPC is traffic retired, not work done.
- Both kernels are L1-resident throughout.
- 2.79×

Original contributions

- **Milstein rather than Euler–Maruyama.** Prior reduced-precision MLMC explores the saving from Euler–Maruyama, on CPUs (Tay [14]) and FPGAs (Nimerenco [7]).
- **Beyond single-asset vanilla options.** An Asian call adds a running time-average rounded every step; a basket adds correlated increments, rounded before they drive any asset.
- **Counter-based generation in this setting,** not previously used for reduced-precision MLMC. Chapter 2 builds an in-register Philox generator [12] and the coupled FP16/FP32 Normal streams [5, 6] the nested estimator requires, benchmarked against Intel MKL.
- **Extending the rounding-error model to Milstein.** Chapter 3 takes the Euler–Maruyama model of Giles and Sheridan-Methven [4], with Higham’s standard floating-point model [8], and extends them to the Milstein increment, identifying which part of the error compensated summation [10] can remove and which it cannot.
- **The complete estimator, analysed on hardware.** Prior work did not develop complete MLMC implementations, and kernels have been studied on hardware before [1] but not the whole nested estimator. Chapter 4 accounts for its speedup in instructions issued and data moved.



References

- [1] N.M. Allayla and S.A. Dawwd. Performance optimization on GPGPU and multicore CPU using roofline model. *IOP Conference Series: Materials Science and Engineering*, 1152:012021, 2021.
- [2] M.B. Giles. Multilevel Monte Carlo path simulation. *Operations Research*, 56(3):607–617, 2008.
- [3] M.B. Giles. Multilevel Monte Carlo methods. *Acta Numerica*, 24:259–328, 2015.
- [4] M.B. Giles and O. Sheridan-Methven. Rounding error using low precision approximate random variables. arXiv:2012.09739, 2020.
- [5] M.B. Giles and O. Sheridan-Methven. Analysis of nested multilevel Monte Carlo using approximate normal random variables. *SIAM/ASA Journal on Uncertainty Quantification*, 10(1): 200–226, 2022.
- [6] M.B. Giles and O. Sheridan-Methven. Approximating inverse cumulative distribution functions to produce approximate random variables. *ACM Transactions on Mathematical Software*, 49(3): 1–29, 2023.
- [7] I.B. Haas and M.B. Giles. A nested MLMC framework for efficient simulations on FPGAs. *Monte Carlo Methods and Applications*, 2025.
- [8] N.J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.
- [9] *Intel Xeon Gold 6538Y Processor (60M Cache, 2.20 GHz) Specifications*. Intel Corporation, 2023.

- [10] W. Kahan. Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1):40, 1965.
- [11] P.E. Kloeden and E. Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, Berlin, 1992.
- [12] J.K. Salmon, M.A. Moraes, R.O. Dror, and D.E. Shaw. Parallel random numbers: as easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'11)*, 2011.
- [13] O. Sheridan-Methven. *Nested multilevel Monte Carlo methods and a modified Euler-Maruyama scheme utilising approximate Gaussian random variables suitable for vectorised hardware and low-precisions*. PhD thesis, University of Oxford, 2021.
- [14] J. Tay. *A Nested Multilevel Monte Carlo Framework with Reduced-Precision Arithmetic on CPUs*. Master's thesis, University of Oxford, 2025.